

czgfx

Graphics Library User Guide

The 8080 graphics library for the CADzilla S-100 graphics board (Hitachi HD63484 ACRTC and Brooktree Bt453 RAMDAC), for programs written with Microsoft M80/L80 under CP/M 2.2

Library version 1.0 · document revision 1.0 · 2026-10-11

Contents

1. Introduction **4**

1.1 What czgfx is . . . 4

1.2 The board, in brief . . . 4

1.3 About this guide . . . 5

2. Getting started **5**

2.1 A project that uses czgfx . . . 5

2.2 A first program . . . 5

3. Conventions **6**

3.1 Coordinates . . . 6

3.2 Registers and arguments . . . 6

3.3 Names . . . 6

3.4 Waiting, and errors . . . 7

4. Setting up the display **7**

4.1 GINIT and the display modes . . . 7

4.2 Clearing . . . 7

4.3 Colors and the palette . . . 7

5. Lines, rectangles and polygons **9**

5.1 The drawing pointer . . . 9

5.2 Polylines and polygons . . . 9

6. Circles, ellipses and arcs **11**

7. Drawing modes **12**

7.1 Operation modes . . . 12

7.2 Color modes . . . 13

7.3 Area modes . . . 13

8. The pattern RAM **14**

8.1 Layout and the pattern pointer . . . 14

8.2 Loading patterns . . . 14

8.3 Line styles . . . 14

8.4 Drawing the pattern itself: GPTN . . . 14

9. PAINT, flood and copy **16**

9.1 Painting areas . . . 16

9.2 Copying blocks . . . 16

10. The overlay **17**

11. Text **19**

11.1 Fonts 19

11.2 Drawing text 20

12. Reading back **21**

13. Library versions **22**

13.1 Moving a program to another release 22

13.2 Making a release 22

Appendix A. Macro reference **23**

Appendix B. Modules and routines **25**

Appendix C. Equates **25**

Appendix D. Building and extending the library **27**

D.1 Adding to the library 27

D.2 M80 and L80 notes 27

Appendix E. Limits **27**

1. Introduction

czgfx is the graphics library for CADzilla programs written in 8080 assembly with Microsoft MACRO-80 (M80) and LINK-80 (L80) under CP/M. It sets up the board and gives a program the whole of the ACRTC's drawing processor through short macros: lines, rectangles, polygons, circles, ellipses and arcs, area fills and copies, the pattern RAM, the operation, color and area modes, a one-bit overlay plane and text in two fonts.

This guide is for programmers using the library. It explains the board's model as far as a program needs it, then each group of calls, with the screens of the **gfxdemo** program (`software/m80/gfxdemo`) as worked examples. The appendices list every macro, routine and equate, and explain how the library itself is built and extended.

1.1 What czgfx is

czgfx comes in two parts:

- `CZGFX.INC`, the header a program INCLUDEs: the board's equates and the `G...` macros.
- `CZGFX.REL`, a LIB80 library of fourteen relocatable modules. L80's library search links in only the modules a program uses, so a program that draws only boxes gets no curve, text or font code (Appendix B lists the modules and their sizes).

Both, with the link script `CZLINK.SUB`, are on the library disk, which a program mounts as CP/M drive `B:`. Each project keeps its own copy of a numbered release of that disk, so a new czgfx version reaches a program only when the program chooses to move to it (chapter 13).

1.2 The board, in brief

CADzilla pairs two chips of the mid 1980s. The **HD63484 ACRTC** (Advanced CRT Controller) generates the display and has a microcoded drawing processor: the host writes 16-bit commands and their parameters into an 8-word FIFO, and the chip draws into its own 2 MB of frame memory, which the CPU never addresses directly. The **Bt453** RAMDAC turns each 8-bit pixel into a color through its 256-entry palette. A separate one-bit **overlay** plane can be shown over the picture in a color of its own. The board decodes eight I/O ports from `70H`:

Port	Name	Use
`70H`	`ACRSR`	ACRTC address register (out) / status (in)
`71H`	`MODEREG`	the board's MODE register: sync polarity, access mode, overlay enable and select (write-only)
`72H`	`ACRDAT`	ACRTC control register data / the command FIFO
`74H`	`DACADR`	Bt453 address register
`75H`	`DACPAL`	Bt453 palette RAM (R, G, B)
`77H`	`DACOVL`	Bt453 overlay colors (R, G, B)

czgfx programs the board for 8 bits a pixel, one screen, at 640x480, 800x600 or 1024x768. A program never needs the ports itself, but the names are in the header if it does.

1.3 About this guide

It describes czgfx 1.0. Macro and routine names, equates, CP/M commands and source are set in `Courier`. "CP" is the ACRTC's current drawing pointer. Paths are relative to the CADzilla repository. The library's own reference, kept with the sources, is `software/m80/czgfx/README.md`.

2. Getting started

2.1 A project that uses czgfx

CADzilla programs are built and run under CP/M 2.2 on the altairsim emulator (or on the real board), each project with its own machine and disks in its `work/` directory. A new M80 project is scaffolded with:

```
.claude/skills/new-cadzilla-project/scripts/new_project.sh myprog \
        --toolchain m80 --mode 1024x768
```

That creates, under `software/m80/myprog/`:

File	What it is
`src/MYPROG.MAC`	a starter program that draws a frame and a box
`machine.toml`	the monitor resolution the project is strapped for (tracked)
`czgfx.version`	the czgfx release the project builds against, e.g. `1.0` (tracked)
`work/`	its altairsim machine: `CPM22SYS.DSK` as `A:`, and its own copy of the pinned library, `CZGFXLIB-1.0.DSK`, as `B:`, read-only (gitignored)

2.2 A first program

A complete program: it sets up the board, draws a frame round the screen and a blue box in the middle, and waits for the drawing to finish.

```
        INCLUDE B:CZGFX.INC     ;equates and macros
;
        CSEG                    ;relocatable -- no ORG
START:  GINIT   CZM1K           ;1024x768, default palette, screen cleared
        GCOLOR  WHITE+BRIGHT
        GBOX    0,0,1023,767    ;a frame round the screen
        GFBOX   448,320,576,448,BLUE+BRIGHT
        GWAIT                   ;Z = drawn cleanly, NZ = command error
        RET                     ;back to CP/M
;
        END     START
```

It is assembled with M80 and linked against the library with the link script on `B:`:

```
A0>M80 MYPROG,MYPROG=MYPROG
No Fatal error(s)
A0>SUBMIT B:CZLINK MYPROG
A0>L80 MYPROG,B:CZGFX/S,MYPROG/N/E
Link-80  3.44  09-Dec-81  Copyright (c) 1981 Microsoft
...
A0>MYPROG
```

`CZLINK.SUB` is `L80 $1,B:CZGFX/S,$1/N$2/E`: the program, then a search of the library (`/S`), writing `MYPROG.COM`. A second argument is passed to L80 as an extra switch: `SUBMIT B:CZLINK MYPROG /M` prints the link map, which shows which library modules were loaded. After a map L80 waits at its `*` prompt;

type `/E` to finish.

A program of several modules assembles each with M80 and links them all before the library: `L80 MAIN,PART2,B:CZGFX/S,MAIN/N/E`.

3. Conventions

3.1 Coordinates

GINIT puts the drawing origin at the **bottom-left** pixel. X runs from 0 to `CZXMAX` left to right, Y from 0 to `CZYMAX` bottom to top. `CZXMAX` and `CZYMAX` are words in the library that GINIT sets for the mode (1023 and 767 at 1024x768); read them as `LHLD CZXMAX##`.

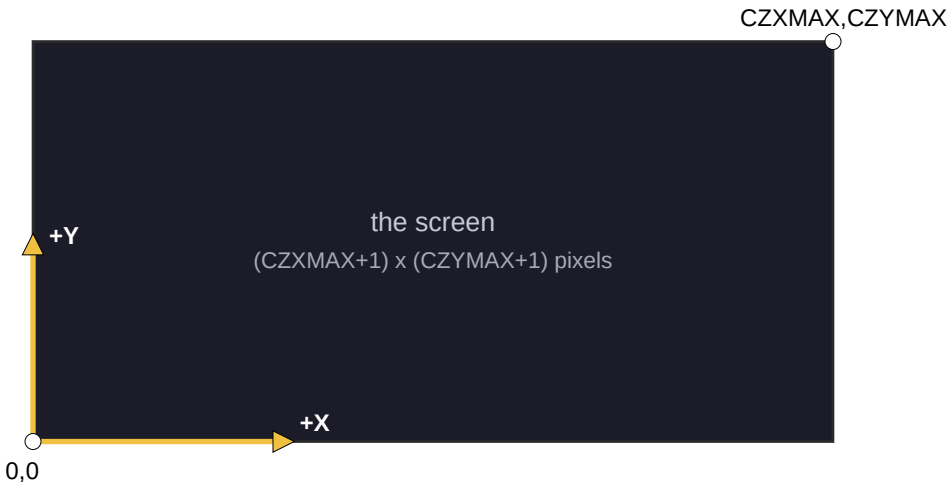


Figure 1. Screen coordinates.

3.2 Registers and arguments

- X goes in HL, Y in DE, a color (palette index) in A.
- Every routine preserves BC, DE and HL unless its entry says otherwise, and clobbers A and the flags. A few macros clobber registers of their own; Appendix A says which.
- A macro argument left **blank** means "already in the register": `GLINE ,100` draws to X = HL, Y = 100, and `GCOLOR` alone takes the color in A. That is how computed values go through the macros.
- The exception: for a shape drawn round a point (`GCIRC`, `GELLIP`, `GPAINT`, `GFLOOD`, `GPTN`) leaving both x and y blank means "at the CP", with no move. `GCIRC` alone is a circle round the CP, radius in HL.
- An argument with spaces or commas in it goes in angle brackets: `GCOLOR <(N MOD 8)*32+BRIGHT>`, `GPOLY <100,100,150,100,125,140>`.

3.3 Names

- Library routines and data outside the macros are written `NAME##`, M80's inline external: `CALL CZLDW##`, `LHLD CZXMAX##`. Don't `EXTRN` library names you don't use: M80 turns every `EXTRN` into a link request, and L80 then loads that module whether anything calls it or not.
- Global names are significant to 6 characters (L80 keeps only the first 6). Every czgfx global is `CZ` and up to four more; don't define `CZ...` names of your own.

- The header also defines, without a prefix, the hue constants (`ORANGE BLUE GREEN CYAN RED MAGENTA YELLOW WHITE` and `BRIGHT`), the opcodes `OP...`, the mode values `OM...`, `CM...` and `AM...`, the copy transforms `CP...`, the port names, the status bits `SR...` and a few more (Appendix C). Don't reuse them in a program.
- The library never calls the BDOS: console I/O is the program's.

3.4 Waiting, and errors

Drawing macros only send commands; the ACRTC draws them from its FIFO. `GWAIT` waits until the FIFO is empty and the last command has ended, and returns the status register in A: **Z** if the drawing ended cleanly, **NZ** on a command error (CER, status bit 7: an undefined command or a bad parameter). A program normally ends each picture with `GWAIT`. `GABORT` stops the chip, flushes the FIFOs and clears CER. On the real board, and in altairsim with `draw_rate = "real"`, drawing takes time, and `GWAIT` is also how a program knows it has finished.

4. Setting up the display

4.1 GINIT and the display modes

`GINIT mode` sets up the whole board for one display mode: the MODE register, the ACRTC's timing and display registers, the default palette, the drawing parameters, the overlay, and a clear screen. The mode is a descriptor in the library:

Descriptor	Mode	Screen
`CZM640`	VESA 640x480 @ 60 Hz	X 0-639, Y 0-479
`CZM800`	VESA 800x600 @ 60 Hz	X 0-799, Y 0-599
`CZM1K`	VESA 1024x768 @ 60 Hz	X 0-1023, Y 0-767

The monitor is fixed-frequency, chosen by a strap (in altairsim, cad0's `mode` in the project's `machine.toml`; 1024x768 by default). The descriptor must match it: the board doesn't reject a mismatch, the picture just lands in the wrong place in the monitor's frame. Always name the descriptor: `GINIT` with no argument means `CZM640`. `GINIT` clobbers all registers.

After `GINIT`: the screen is color 0; the drawing color (CL1) is 255, white; CL0 and CCMP are 0; the drawing mode is replace with no area checking; the pattern RAM is all ones; the area is the whole screen; the overlay is clear, hidden and white; and drawing goes to the frame.

4.2 Clearing

`GCLS [c]` clears the whole screen to color c (default 0) with the ACRTC's block clear, the fastest way to fill the screen. `GOVCLS` clears the overlay (chapter 10).

4.3 Colors and the palette

A pixel is a palette index, 0-255. `GCOLOR c` sets the drawing color. GINIT loads a default palette of eight hues with 32 brightness levels each: **index = hue + level**, where the hue is one of the constants below and the level runs from 0 (black) to `BRIGHT` (31). `GCOLOR CYAN+BRIGHT` is full cyan; `GCOLOR RED+16` a half-bright red.

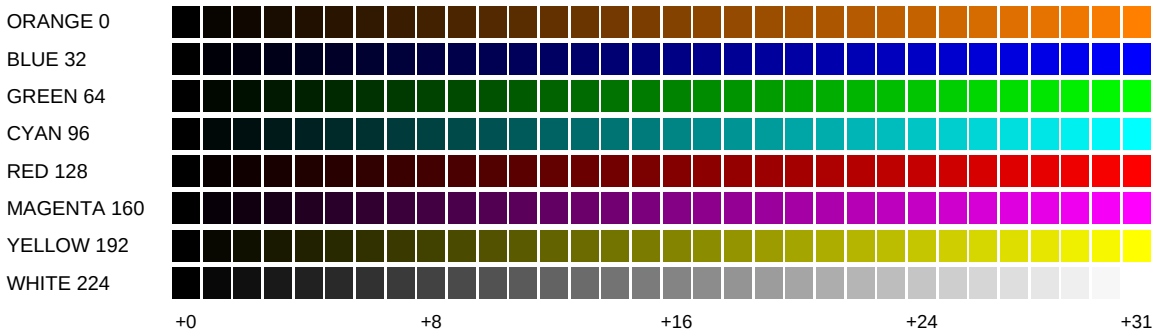
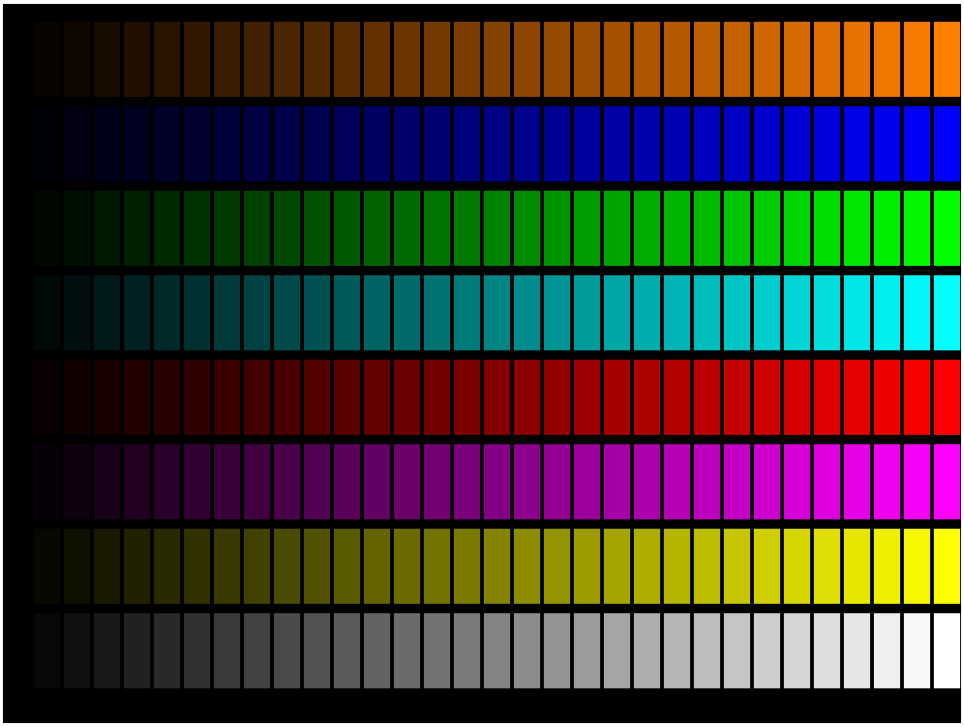


Figure 2. The default palette, as GINIT loads it.

`GPAL i,r,g,b` sets palette entry i to any color (0-255 each component, clobbers BC and E). Changing an entry recolors everything already drawn in it at once, which makes palette animation cheap. `CALL CZPALD##` loads the default palette again.



gfxdemo screen 1: all 256 entries of the default palette as filled boxes (GFBOX), in a white frame (GBOX).

5. Lines, rectangles and polygons

5.1 The drawing pointer

The ACRTC draws from its current drawing pointer, the CP. `GMOVE x,y` moves it; the drawing macros draw from it and most leave it at their end point. Each comes in an absolute form, to x,y, and a relative one, by dx,dy from the CP:

Absolute	Relative	Draws	CP afterwards
`GMOVE x,y`	GRMOVE dx,dy	nothing: moves the CP	x,y
`GLINE x,y`	GRLINE dx,dy	a line from the CP, its end point **not** drawn	x,y
`GRECT x,y`	GRRECT dx,dy	a rectangle outline, corners CP and x,y	unchanged
`GFRECT x,y`	GRFRCT dx,dy	a filled rectangle, both corners included	one row past the far edge
`GDOT x,y`		one pixel	x,y

Because a line's end point isn't drawn, a closed shape drawn from lines lights each corner exactly once, which matters when drawing in EOR mode. Three macros combine a move with a drawing:

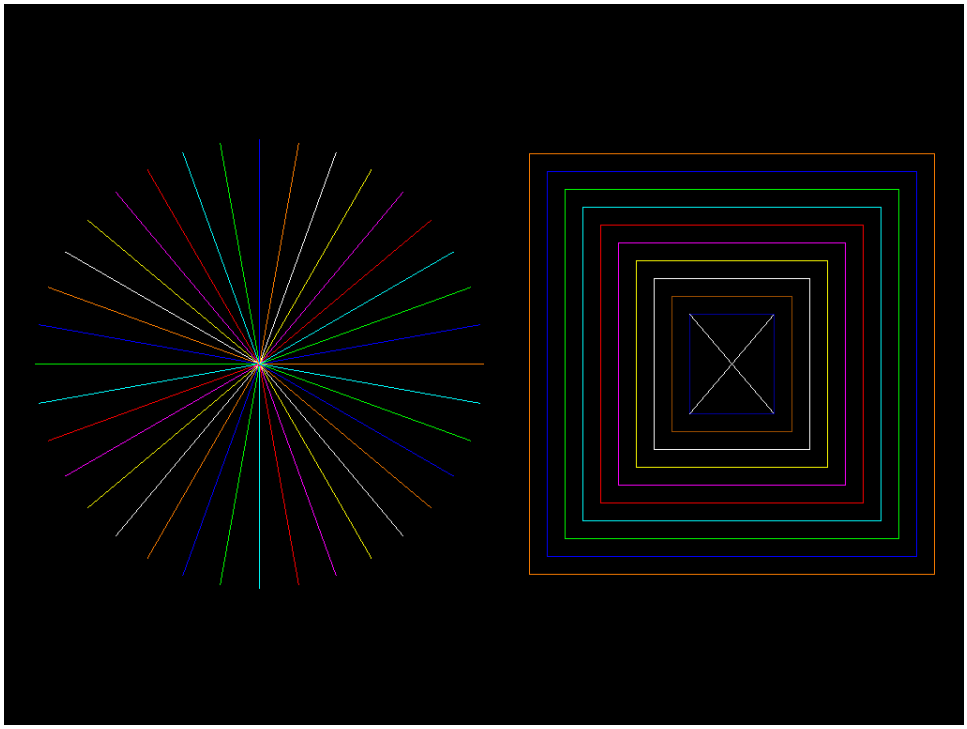
Macro	Does
`GSEG x1,y1,x2,y2`	a line from x1,y1 to x2,y2 (x2,y2 not drawn)
`GBOX x1,y1,x2,y2`	a rectangle outline with corners x1,y1 and x2,y2
`GFBOX x1,y1,x2,y2[,c]`	a filled rectangle, in color c if given

5.2 Polylines and polygons

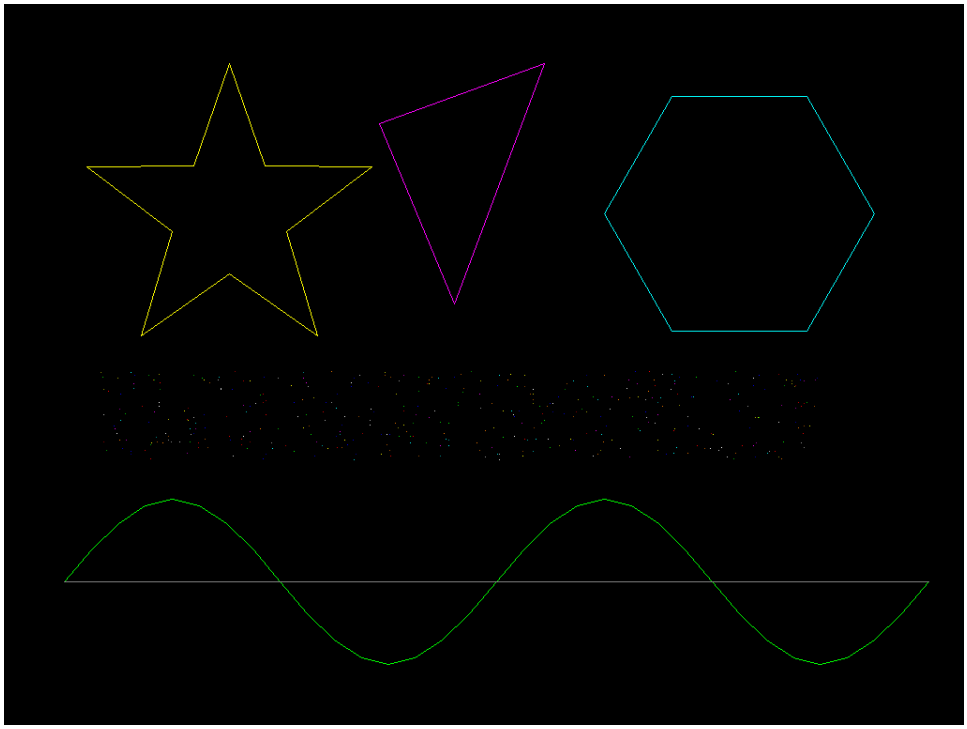
A polyline is drawn through a list of points, a polygon is closed back to its first point. The points can be inline, or in a named table built once in the data:

```
GPOLY   <240,704,202,595,88,594,179,525,146,414>   ;inline
GPLG    TRI                                        ;from a table
...
GPTS    TRI,<480,448,576,704,400,640>              ;the table
```

`GPLINE` and `GPOLY` place their points inline (and jump over them); `GPLT tbl` and `GPLG tbl` draw from a table. A table is `DW` n followed by n X,Y pairs; `GPTS` builds one. A list too long for one source line can be written out with `DW` by hand in the same layout. At least 2 points; these macros clobber HL.



gfxdemo screen 2: a 36-spoke starburst from a data table (GMOVE, GLINE), and nested rectangles unrolled at assembly time with REPT (GBOX).

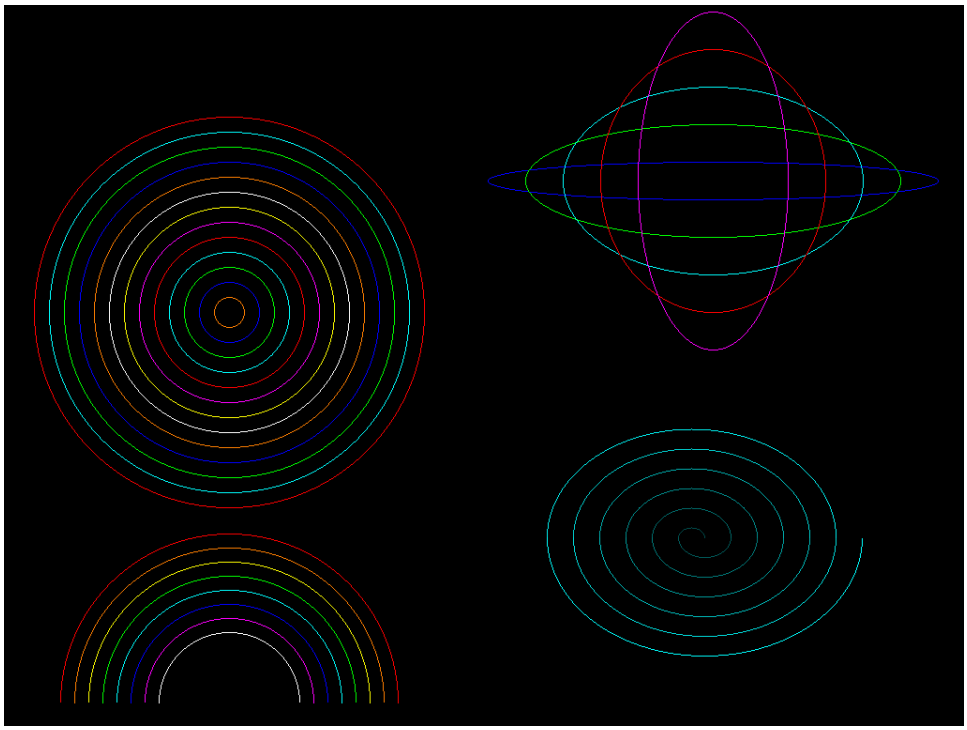


gfxdemo screen 3: a star and a hexagon (GPOLY), a triangle from a table (GPTS, GPLG), a sine wave (GPLT) and 400 random dots (GDOT).

6. Circles, ellipses and arcs

Macro	Draws
`GCIRC x,y,r`	a circle of radius r centred on x,y; the CP is left at the centre
`GELLIP x,y,rx,ry`	an ellipse with semi-axes rx (along X) and ry (along Y), each up to 4095
`GARC xc,yc,xe,ye[,DIRCW]`	an arc from the CP round the centre xc,yc to where it meets the ray from the centre through xe,ye
`GRARC`	the same, the centre and xe,ye relative to the CP
`GEARC rx,ry,xc,yc,xe,ye[,DIRCW]`	an arc of an ellipse whose semi-axes are in the ratio rx:ry
`GREARC`	the same, relative to the CP

- An arc starts at the CP, and its radius is the CP's distance from the centre. It runs counter-clockwise, or clockwise with `DIRCW`, and stops at the ray through xe,ye, so xe,ye need not be exactly on the curve. The CP is left at xe,ye, which is not drawn.
- For an elliptical arc only the ratio rx:ry matters (each at most 90): the size comes from the CP. Started on the ellipse's horizontal axis, a distance d from the centre, the arc lies on the ellipse with semi-axes d along X and d * ry / rx along Y.
- Circles and ellipses are drawn in order round the curve, so a dashed line style runs along them (chapter 8).
- The arc macros clobber HL.



gfxdemo screen 4: concentric circles (GCIRC), a rainbow of half-circle arcs drawn both ways round (GARC, DIRCW), ellipses (GELLIP), and a spiral of elliptical half arcs (GEARC).

7. Drawing modes

Every ACRTC drawing command carries a mode in its low byte. czgfx keeps that byte, `CZDM`, and adds it to every drawing command it sends (all but the moves). `GINIT` resets it to 0: replace, two-color pattern, no area checking.

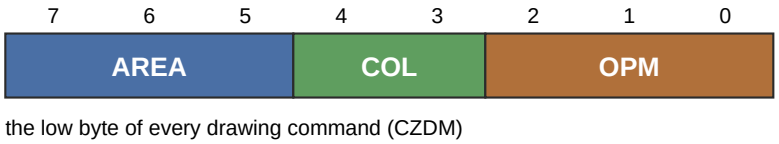


Figure 3. The drawing mode byte.

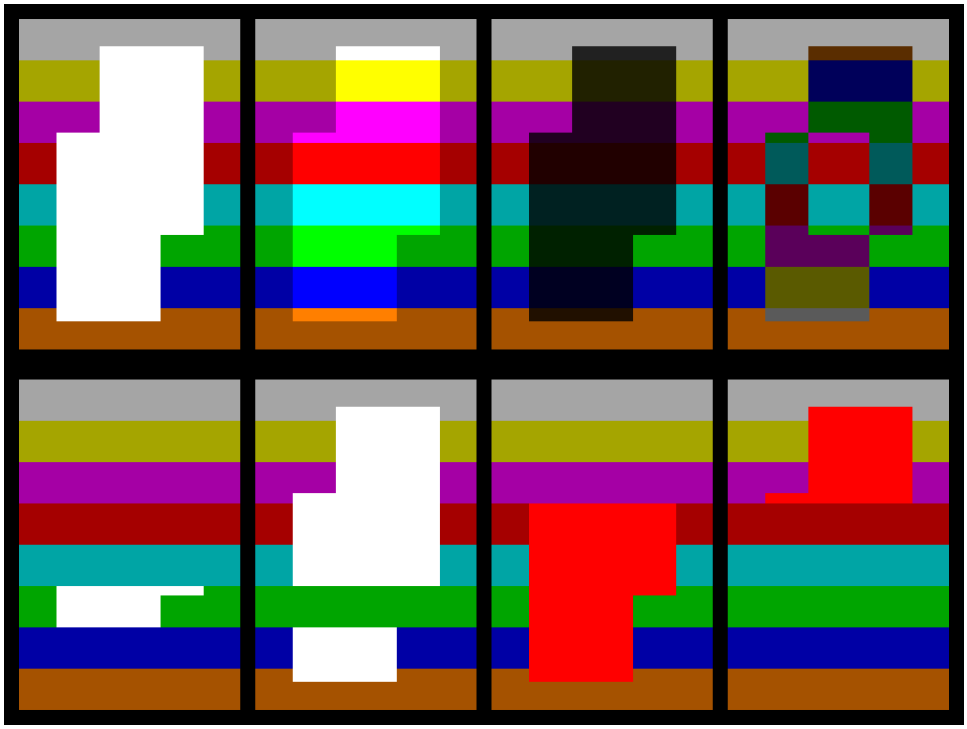
`GMODE m` sets the whole byte, e.g. `GMODE OMEOR+CMCL1+AMCLIP`; `GMODE` alone resets it. `GOPM`, `GCMODE` and `GAMODE` each set one field and keep the others.

7.1 Operation modes

The operation mode says how the color C being drawn meets the pixel P already there:

Value	Result
`OMREP`	C (replace)
`OMOR`	P OR C
`OMAND`	P AND C
`OMEOR`	P XOR C: drawing the same thing twice restores the picture
`OMEQ`	C only where P = CCMP (set with GCCMP c)
`OMNE`	C only where P is not CCMP
`OMLT`	C only where P < C
`OMGT`	C only where P > C

The logic and comparisons work on the palette index.



gfxdemo screen 5: in each panel, two overlapping boxes drawn over colored bands in one operation mode. Top: replace, OR, AND, EOR (where the boxes overlap, EOR restores the bands). Bottom: =CCMP, <>CCMP, C.

7.2 Color modes

The ACRTC draws every pixel through its pattern RAM (chapter 8). The color mode says what each pattern bit draws. With GINIT's all-ones pattern, every mode but CMCL0 simply draws in CL1.

Value	A 1 bit draws	A 0 bit draws
`CMSEL`	CL1 (GCOLOR)	CL0 (GBCOL)
`CMCL1`	CL1	nothing: the pattern is transparent
`CMCL0`	nothing	CL0
`CMPRAM`	the pattern RAM is a 4x4 tile of colors (GTILE)	

7.3 Area modes

The area modes check each pixel against a rectangle, set with `GAREA x1,y1,x2,y2` (low and high corners, both inside) and reset to the whole screen with `GAFULL`:

Value	Effect
`AMNONE`	no checking
`AMCLIP`	draw only inside the area
`AMHOLE`	draw only outside it
`AMCLPD / AMHOLD`	as AMCLIP / AMHOLE, and set ARD if a pixel was dropped
`AMSTOP`	stop at the first pixel outside the area, and set ARD
`AMSTPI`	stop at the first pixel inside the area, and set ARD

ARD is status bit 6: after GWAIT, ANI SRARD tests it. After a stop the CP is the pixel where the drawing stopped; GGETCP reads it back into HL and DE, and also clears ARD.

8. The pattern RAM

The ACRTC draws every pixel, of lines and fills alike, through its 16-word pattern RAM. GINIT fills it with ones, so everything draws in CL1 until a program loads a pattern of its own.

8.1 Layout and the pattern pointer

- In the bit color modes word n is pattern **row n** and bit m its **column m**. Row 0 is the first row drawn (the bottom of a fill), bit 0 the leftmost pixel.
- A pattern pointer walks the RAM: one bit per pixel along a line (continuing round the corners of a rectangle, polygon or circle), one row per raster in a fill.
- GPSCAN x0,y0,x1,y1[,zx,zy] sets the window it walks, columns x0..x1 and rows y0..y1, and optionally a zoom that draws each bit zx+1 times across and zy+1 times up. It also puts the pointer at the window's start.
- The pointer is **live**: it stays where the last drawing left it. Reset it with GPPTR x,y before a fill whose tiling must line up. GPDEF puts everything back as GINIT left it.

8.2 Loading patterns

```
GLDPAT  RING            ;load a 16x16 pattern from row 0
GPSCAN  0,0,15,15       ;the whole pattern, pointer at 0,0
GCMODE  CMCL1           ;1 bits in CL1, 0 bits transparent
GFBOX   100,100,300,200
...
GPAT    RING,<0,0,07E0H,1FF8H,1E78H,381CH,381CH,300CH,...>
```

GPAT name,<w0,...> defines a pattern table (DW n, then n words); GLDPAT tbl[,pra] loads it from pattern word pra (default 0). GTILE name,<c0,...,c15> defines a 4x4 color tile for CMPRAM: 16 palette indexes, c0..c3 its bottom row.

8.3 Line styles

GLSTYL pat[,zoom] is the quick way to dashed lines: pattern row 0 becomes the 16-bit style (bit 0 first), the window becomes 16x1 and the pointer is reset. Fills use the style too, so set it back with GLSTYL 0FFFFH (or GPDEF) afterwards. In CMCL1 the gaps leave the picture alone; in CMSEL they draw in CL0.

```
GCMODE  CMCL1
GLSTYL  0F0F0H          ;4 on, 4 off
GBOX    40,24,200,184   ;the dashes run on round the corners
GLSTYL  0FFFFH          ;solid again
```

8.4 Drawing the pattern itself: GPTN

GPTN x,y,w,h[,dir] draws the pattern RAM as a w x h picture with its corner at x,y. Its rows run in direction dir, 45 degrees apart counter-clockwise from +X, and are stacked 90 degrees further round; adding PTSLNT stacks them only 45 degrees round, slanting the picture. A diagonal direction leaves gaps between pixels, as the chip's PTN does. Text (chapter 11) is drawn this way.

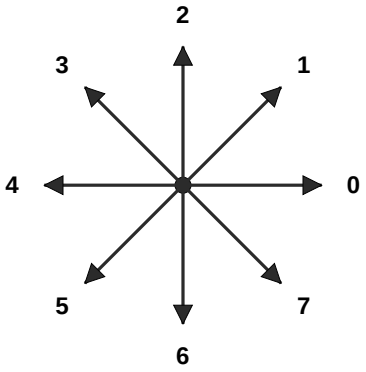
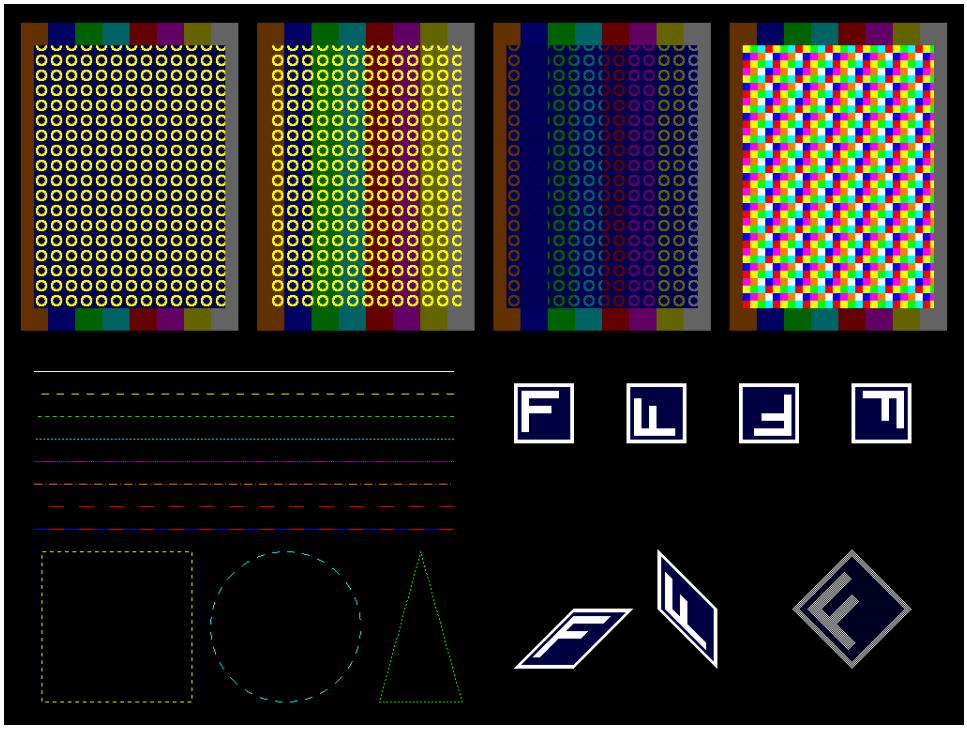


Figure 4. Directions 0-7, used by GPTN and text.



gfxdemo screen 6. Top: a ring pattern filled in CMSEL, CMCL1 and CMCL0, and a 4x4 color tile in CMPRAM. Bottom left: GLSTYL line styles, and dashes running round a box, a circle and a triangle. Bottom right: GPTN turned 0, 90, 180 and 270 degrees, slanted, and at 45 degrees.

9. PAINT, flood and copy

9.1 Painting areas

`GPAINT x,y,e` fills outward from x,y, in the current color, pattern and modes, up to pixels of the edge color e. `GFLOOD x,y,c` instead fills the connected region of color c round x,y, recoloring one shape.

- **CL0 and CL1 are edges too.** The chip also stops at any pixel already in CL0 or CL1. After GINIT CL0 is 0, so painting over black paints nothing until `GBCOL` sets CL0 to something else. For a solid fill, set it to the fill color.
- **Outlines must be closed.** The fill is 4-connected, so an 8-connected outline (lines, circles) holds it, but a line's end point isn't drawn: run spokes a pixel past a circle.
- **On the real board**, PAINT keeps only four pending seed points and hands any more to the host, which czgfx doesn't do, so a complicated area may be left partly unfilled. altairsim has no such limit.

9.2 Copying blocks

`GCOPY x1,y1,x2,y2,xd,yd[,t]` copies the rectangle with corners x1,y1 and x2,y2 so that corner x1,y1 lands on xd,yd, transformed by t. The operation and area modes apply at the destination; the color mode doesn't, since the source pixel is the color. The macro clobbers HL.

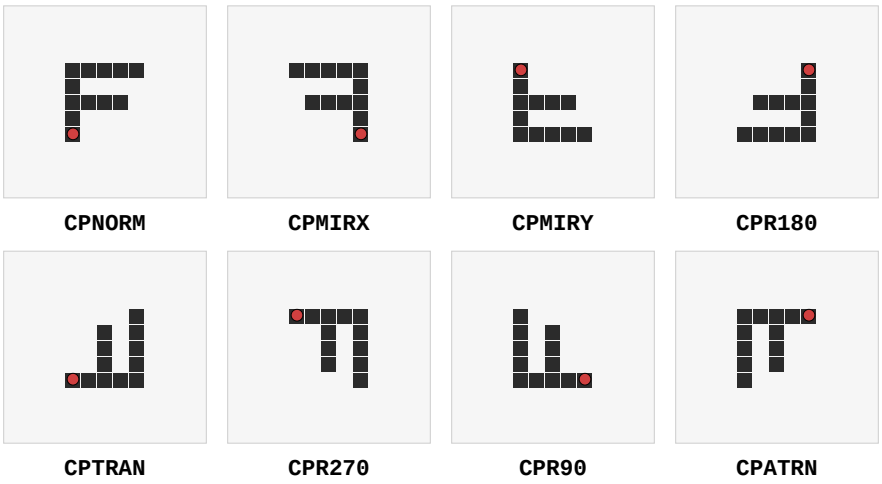
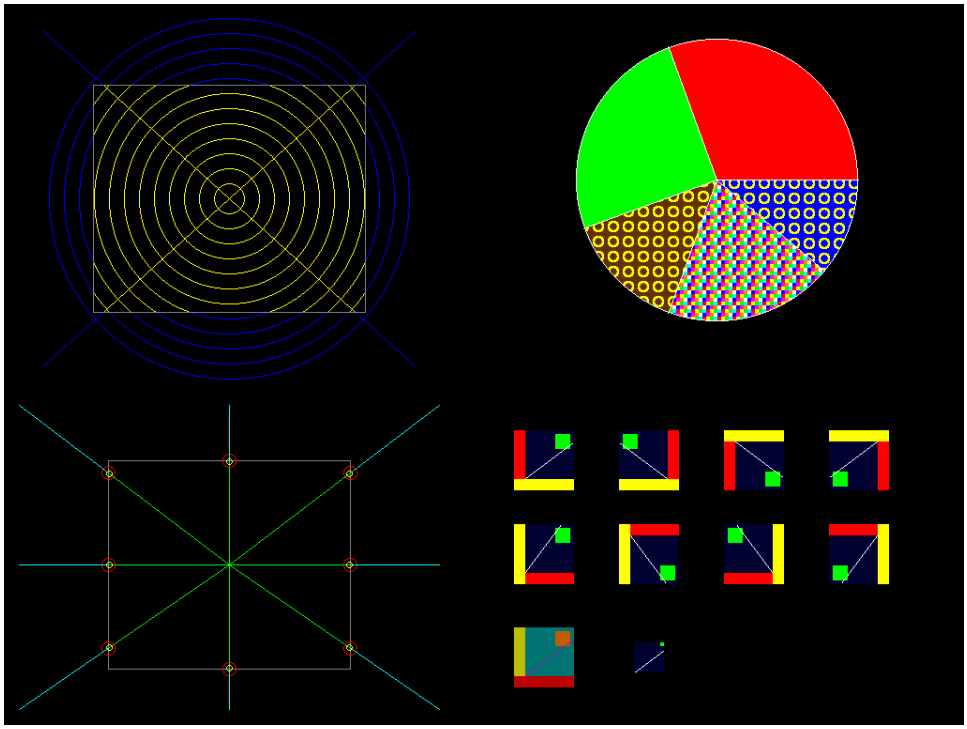


Figure 5. GCOPY's transforms, shown on an F. The red dot is the source corner x1,y1, which lands on xd,yd.



gfxdemo screen 7. Top left: circles clipped to an area (AMCLPD, yellow) and to outside it (AMHOLD, blue). Bottom left: lines that stop at the edge of an area (AMSTOP, AMSTPI), ringed where GGETCP found them. Top right: a pie chart painted solid, patterned and flood-recolored. Bottom right: a picture copied in all eight orientations, in EOR, and clipped.

10. The overlay

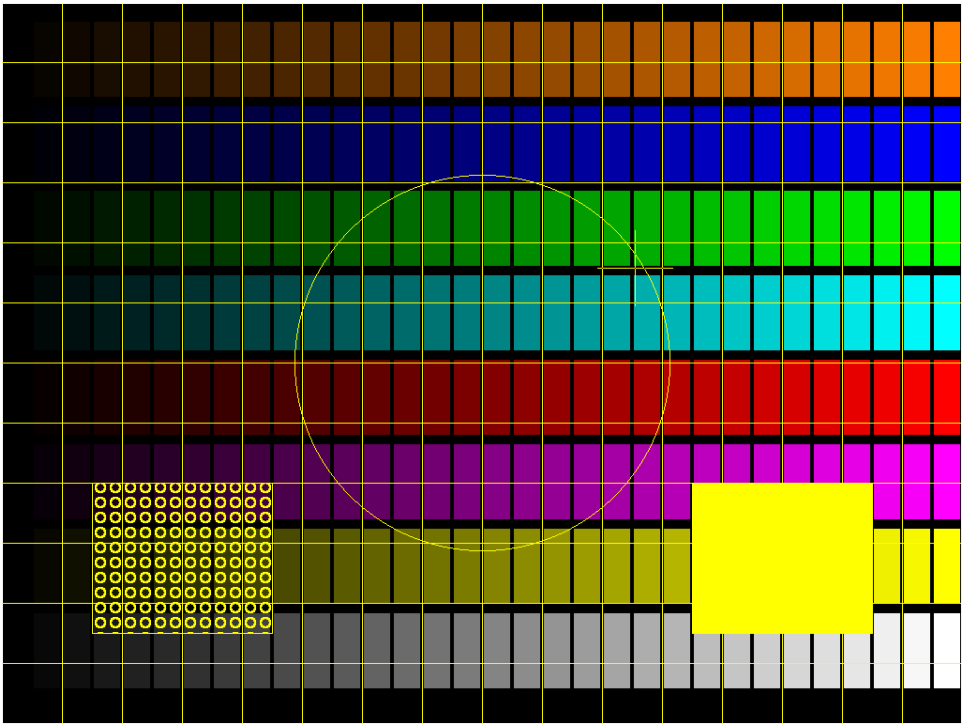
The board has a second plane over the 8-bit picture: the **overlay**, one bit per pixel in its own memory. Wherever a bit is set and the overlay is shown, the Bt453 shows the overlay color instead of the pixel's palette color; a clear bit lets the picture through. So what is drawn on the overlay can be shown, hidden, recolored or erased without disturbing the picture under it: a cursor, a grid, selection marks, rubber-band lines, text.

Macro	Does
`GOVON / GOVOFF`	show / hide the overlay; what is drawn in it stays
`GOVPAL r,g,b`	the overlay color (clobbers BC, E)
`GOVL`	draw on the overlay from here on
`GFRAME`	draw on the frame again
`GOVCLS`	clear the whole overlay, from either

- Between `GOVL` and `GFRAME` every drawing macro draws on the overlay, with the frame's own coordinates.
- Overlay pixels are one bit, so the only colors are `OVSET` (on) and `OVCLR` (off): `GCOLOR OVSET` draws, `GCOLOR OVCLR` erases.
- `OMEOR` toggles bits: drawing a shape twice undraws it, the classic moving cursor.
- Patterns, the area modes and `GPAINT` work on the overlay (set `GBCOL OVSET` before painting); `CMPRAM` tiles mean nothing at one bit a pixel. `GCOPY` copies within the overlay, not between the planes.

- GOVL and GFRAME wait for the ACRTC to finish, since they switch its pixel depth, and leave the CP at 0,0: GMOVE afterwards.

```
GOVPAL  255,255,0       ;yellow
GOVL                    ;draw on the overlay
GCOLOR  OVSET
GMODE   OMEOR
CALL    XHAIR           ;draw a crosshair...
CALL    XHAIR           ;...and undraw it: the picture is untouched
GMODE
GFRAME                  ;back to the frame
GOVON                   ;show the overlay
```



gfxdemo screen 8: the palette of screen 1 on the frame, and on the overlay in yellow a grid, a circle, a patterned and a solid box, and a crosshair moved in EOR. Hiding the overlay (GOVOFF) shows screen 1 unchanged.

How it works: the overlay has no display cycles of its own; each frame fetch also reads the overlay word for the same 16 pixels. The ACRTC draws it through a character (CHR) screen that the board routes to the overlay memory and never displays; GINIT sets it up with the frame's coordinates. GOVL switches the chip to 1 bit a pixel and GFRAME back to 8, after waiting, so that queued commands run at the depth they were sent for.

11. Text

czgfx draws text a character at a time with the ACRTC's PTN command: each glyph is unpacked into the pattern RAM and PTN draws the character cell at the pen. That brings scaling (the pattern zoom, 1-16 times), turning (PTN's direction, in 45-degree steps) and the drawing modes with it. Text goes on whichever plane is being drawn, frame or overlay, in CL1.

11.1 Fonts

A font is chosen by naming it with `GFONT`, and naming it is also what links it in, so a program carries only the fonts it uses. Both cover characters 20H-7EH; others draw as a blank.

Font	Cell	Glyphs	Bytes
`CZF8`	8x8	5x7, no true descenders	480
`CZF11`	9x11	7x10: capitals 8 rows, 2 descender rows	860

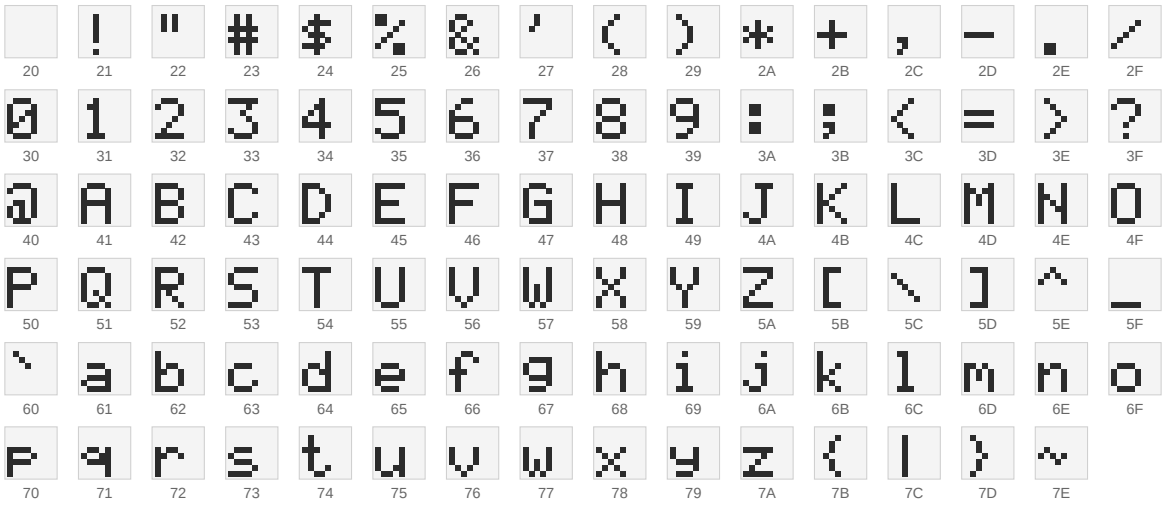


Figure 6. CZF8, the 8x8 font.

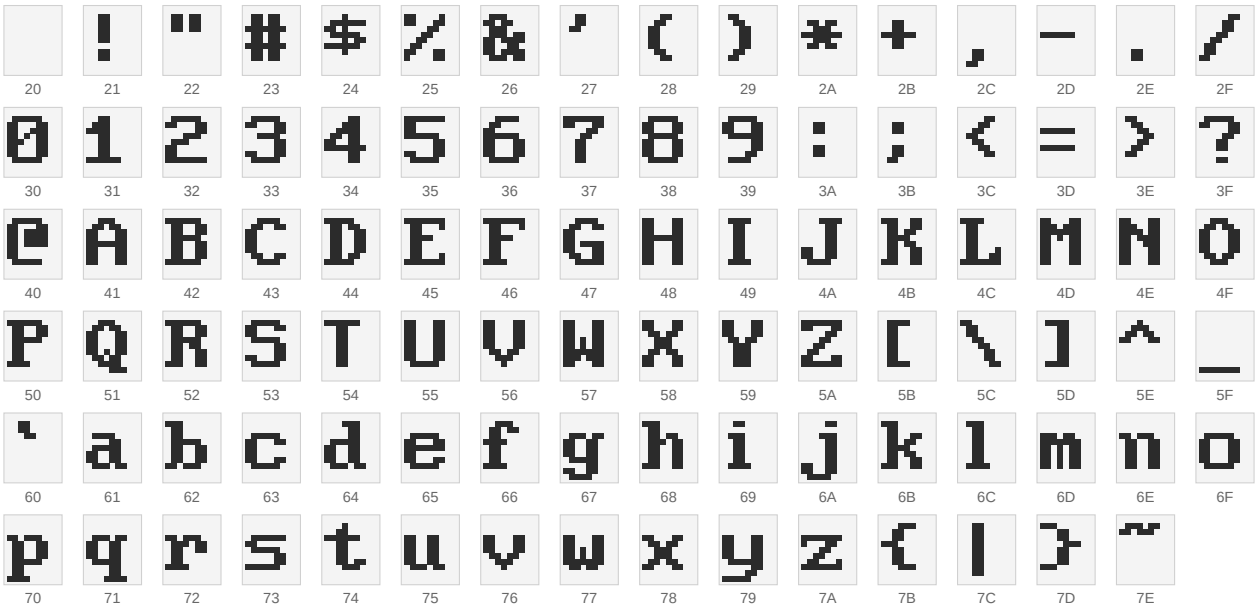


Figure 7. CZF11, the 9x11 font.

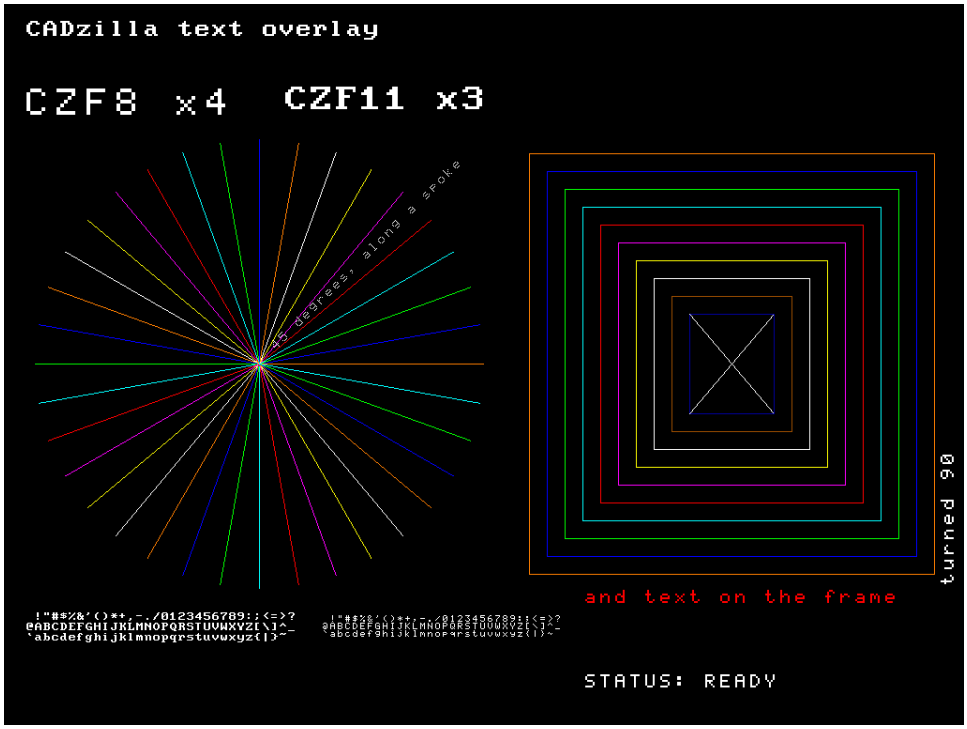
11.2 Drawing text

Macro	Does
`GFONT f`	draw text in font f, CZF8 or CZF11 (clobbers HL)
`GTPOS x,y`	put the pen at x,y: the bottom-left corner of the next character's cell; also the start of the line
`GTATTR s[,d[,TXOPAQ]]`	scale 1-16, direction 0-7, and opaque cells (clobbers BC)
`GTEXT <'text'>`	draw the text, placed inline, at the pen (clobbers HL, B)
`GTSTR`	draw the B characters at HL
`GTCHR [c]`	draw one character (blank: in A)
`GTNL`	a new line: the start of the line moves one line on, and the pen goes there

```
GOVL                    ;a title on the overlay
GCOLOR  OVSET
GFONT   CZF11
GTATTR  2               ;scale 2, along +X
GTPOS   24,730
GTEXT   <'CADzilla text overlay'>
GTNL                    ;the next line, under it
GFRAME
```

- After each character the pen moves one cell width times the scale along the direction; a string is characters x cell width x scale long. `GTNL` moves one cell height times the scale, 90 degrees clockwise from the direction (down, for ordinary text).
- Both fonts have a blank top row and blank right-hand columns, so text set at the cell pitch doesn't touch.
- **Transparent** text (the default) leaves the picture alone in a glyph's 0 bits. **Opaque** text (`TXOPAQ`) draws the whole cell, the 0 bits in CL0 (`GBCOL`): for text rewritten in place, such as a status line. On the overlay, use `GBCOL OVCLR`.

- The color mode is forced only while a string is drawn; the operation and area modes apply as usual.
- A string leaves the pattern RAM as GINIT does, and moves the CP: load your own pattern, and `GMOVE`, afterwards.
- Choose a font first: until `GFONT`, `GTEXT`, `GTSTR` and `GTCHR` return C and draw nothing.



gfxdemo screen 10: screen 2 on the frame, and on the overlay a 9x11 title at scale 2, both fonts scaled up, both character sets (three lines each, GTNL), text turned 90 and 45 degrees, and an opaque status line rewritten in place. The red line is drawn on the frame itself.

Size: the string-drawing core (`CZTXT`, 444 bytes) is all a program that only draws strings needs; the pen, attribute, character and newline calls are a separate module (`CZTXL`). A program that wants to stay small can store the pen, scale and direction into the text state `CZTST` itself, at offsets +0/+2 (pen X, Y), +8 (scale), +9 (direction) and +10 (flags), and call `CZTFNT` and `CZTSTR` directly, as CZBASIC does.

12. Reading back

Macro	Returns
`GGETCP`	the CP: HL = X, DE = Y. It also clears ARD.
`GRDOT x,y`	the pixel at x,y: A = its palette index, NC; or C (A = 0) if x,y is off the screen

Both read through the ACRTC's read FIFO, after everything sent before them has been drawn. `GRDOT` reads the frame, whichever plane is being drawn.

13. Library versions

czgfx is released in numbered versions, and each program builds against the release it chooses, from its own copy. A new czgfx version therefore reaches a program only when the program moves to it, and a program's `work/` directory is self-contained.

Where	What
`CZGFX.INC: CZVERS`	the version: `CZVERS EQU 0100H` is 1.0, major in the high byte, minor in the low. A program can check it at assembly time.
`software/m80/czgfx/releases/CZGFXLIB-<v>.DSK`	the releases: library disks, one per version, tracked and never changed
`software/<toolchain>/<project>/czgfx.version`	the pin: one line, e.g. `1.0`, or `dev` for the current, unreleased build
`software/<toolchain>/<project>/work/CZGFXLIB-<v>.DSK`	the project's own copy of its pinned release, mounted as `B:`, read-only

13.1 Moving a program to another release

```
echo 1.1 > software/m80/myprog/czgfx.version
tools/altairsim/scripts/attach_libdisk.sh software/m80/myprog/work
```

then rebuild and relink the program. `attach_libdisk.sh` copies the pinned release into `work/`, mounts it, and removes copies of releases no longer pinned. A program's `.COM` keeps the library code it was linked with until it is relinked.

13.2 Making a release

Raise `CZVERS` in `CZGFX.INC`, rebuild the library (Appendix D), then run `software/m80/czgfx/tools/release.sh`. It snapshots the build disk as `releases/CZGFXLIB-<major>.<minor>.DSK`, read-only, and refuses to overwrite an existing release or to release a build disk older than its sources. Add the new release to git.

Appendix A. Macro reference

Macro	Does
`GINIT [mode]`	set up the board for CZM640 (default), CZM800 or CZM1K; clear. Clobbers all
`GCLS [c]`	clear the screen to c (default 0)
`GCOLOR [c]`	drawing color (CL1)
`GBCOL [c]`	background color CL0
`GCCMP [c]`	compare color for OMEQ/OMNE
`GPAL i,r,g,b`	set a palette entry. Clobbers BC, E
`GWAIT`	wait for the drawing to finish: A = status, Z = OK
`GABORT`	abort: flush the FIFOs, clear CER
`GMOVE x,y GRMOVE dx,dy`	move the CP
`GLINE x,y GRLINE dx,dy`	line from the CP (end not drawn)
`GRECT x,y GRRECT dx,dy`	rectangle outline from the CP
`GFRECT x,y GRFRCT dx,dy`	filled rectangle from the CP
`GDOT x,y`	one pixel
`GSEG x1,y1,x2,y2`	a line
`GBOX x1,y1,x2,y2`	a rectangle outline
`GFBOX x1,y1,x2,y2[,c]`	a filled rectangle
`GPLINE <pts> GPOLY <pts>`	polyline / polygon, points inline. Clobbers HL
`GPTS name,<pts>`	define a point table
`GPLT tbl GPLG tbl`	polyline / polygon from a table
`GCIRC x,y,r`	circle
`GELLIP x,y,rx,ry`	ellipse
`GARC xc,yc,xe,ye[,DIRCW] GRARC`	circular arc from the CP. Clobbers HL
`GEARC rx,ry,xc,yc,xe,ye[,DIRCW] GREARC`	elliptical arc from the CP. Clobbers HL
`GMODE [m]`	the whole drawing mode
`GOPM op GCMODE cm GAMODE am`	one field of the drawing mode
`GAREA x1,y1,x2,y2 GAFULL`	the area / the whole screen
`GGETCP`	read the CP: HL, DE. Clears ARD
`GRDOT x,y`	read a pixel: A; C if off the screen
`GPAT name,<w...> GTILE name,<c...>`	define a pattern / color tile
`GLDPAT tbl[,pra]`	load the pattern RAM
`GPSCAN x0,y0,x1,y1[,zx,zy]`	pattern window and zoom. Clobbers HL
`GPPTR x,y`	pattern pointer. Clobbers HL
`GPDEF`	pattern RAM, window and pointer as after GINIT
`GLSTYL pat[,zoom]`	line style

Macro	Does
`GPTN x,y,w,h[,dir]`	draw the pattern RAM
`GPAINT x,y,e`	paint out to edge color e
`GFLOOD x,y,c`	recolor the region of color c
`GCOPY x1,y1,x2,y2,xd,yd[,t]`	copy a block. Clobbers HL
`GOVON GOVOFF`	show / hide the overlay
`GOVPAL r,g,b`	overlay color. Clobbers BC, E
`GOVL GFRAME`	draw on the overlay / the frame
`GOVCLS`	clear the overlay
`GFONT f`	choose a font, CZF8 or CZF11. Clobbers HL
`GTPOS x,y`	text pen
`GTATTR s[,d[,TXOPAQ]]`	text scale, direction, opaque. Clobbers BC
`GTEXT <'text'>`	draw inline text. Clobbers HL, B
`GTSTR GTCHR [c]`	draw B characters at HL / one character
`GTNL`	new line

Appendix B. Modules and routines

The routines behind the macros, by module, with each module's size in czgfx 1.0. Call them as NAME##. A program links only the modules it uses, plus the ones those call.

Module	Bytes	Routines and data
CZCORE	167	CZWREG (B = register, HL = value), CZPUTW (HL to the FIFO), CZWAIT, CZXY / CZXYM (A = opcode, HL, DE), CZCMD (A = opcode, with the mode), CZWPR (A = drawing register, HL), CZSEND (HL = table, BC = words), CZRTAB, CZLDW, CZABT; data CZDM
CZINIT	566	CZINIT (HL = descriptor), CZCLS (A), CZOCLS, CZPDEF; data CZM640 CZM800 CZM1K, CZXMAX, CZYMAX, CZMREG (the MODE register), CZDPB
CZPAL	129	CZPALD (default palette), CZPALS (A = index, B/C/E = R/G/B)
CZDRAW	66	CZCOL, CZMOVE, CZLINE, CZRECT, CZFRCT, CZDOT, CZRMOV, CZRLIN, CZRRCT, CZRFRC
CZPOLY	49	CZPLL, CZPLG (HL = point table)
CZCURV	192	CZCRCL (HL = r), CZELPS (HL = rx, DE = ry), CZARC (A = OPAARC/OPRARC/OPAEAR/OPREAR + DIRCW, HL = table)
CZFILL	51	CZPANT (A = edge), CZFLOD (A = region), CZGCPY (A = transform, +10H relative; HL = table)
CZPAT	149	CZLPAT (A = word, HL = table), CZLSTY (HL = style, A = zoom), CZPTN (HL = size, A = direction), CZPSCN (HL = 6-byte table)
CZMODE	246	CZOPM, CZCMOD, CZAMOD (A), CZCOL0, CZCCMP (A), CZAMIN, CZAMAX (HL, DE), CZAFUL, CZGCP, CZRDOT
CZOVL	78	CZOVON, CZOVOF, CZOPAL (B/C/E = R/G/B), CZOVL, CZFRM
CZTXT	444	CZTFNT (HL = font), CZTSTR (HL = text, B = length); data CZTST
CZTXL	149	CZTPOS (HL, DE), CZTATR (A = scale, B = direction, C = flags), CZTCHR (A), CZTNL
CZFNT8	480	data CZF8, the 8x8 font
CZFN11	860	data CZF11, the 9x11 font

Appendix C. Equates

The header defines, besides the macros:

Group	Names
Ports	CZBASE (70H), ACRSR, MODEREG, ACRDAT, DACADR, DACPAL, DACOVL
Status bits	SRWFE 01H, SRWFR 02H, SRRFR 04H, SRCED 20H, SRARD 40H, SRCER 80H
MODE bits	MDOLEN 08H (overlay shown), MDOLSL 10H
Hues	ORANGE 0, BLUE 32, GREEN 64, CYAN 96, RED 128, MAGENTA 160, YELLOW 192, WHITE 224; BRIGHT 31
Operation modes	OMREP OMOR OMAND OMEOR OMEQ OMNE OMLT OMGT (0-7)
Color modes	CMSEL 00H, CMCL1 08H, CMCL0 10H, CMPRAM 18H
Area modes	AMNONE 00H, AMSTOP 20H, AMCLIP 40H, AMCLPD 60H, AMSTPI 0A0H, AMHOLE 0C0H, AMHOLD 0E0H
Copy transforms	CPNORM CPMIRY CPMIRX CPR180 CPTRAN CPR270 CPR90 CPATRN (0-7)

Group	Names
Directions	`DIRCCW 0, DIRCW 1 (curves); PTSLNT 8 (GPTN)`
Overlay, text	`OVSET 0FFH, OVCLR 0; TXOPAQ 1`
Opcodes	`OPMOVE OPRMOV OPLINE OPRLIN OPRECT OPRRCT OPAPLL OPAPLG OPCRCL OPELPS OPAARC OPRARC OPAEAR OPREAR OPFRCT OPRFRC OPPANT OPDOT OPPTN OPAGCP OPRGCP OPRD (command high bytes); CMDDOT`
Version	`CZVERS (0100H for 1.0)`

Appendix D. Building and extending the library

The library is built on czgfx's own altairsim machine (`software/m80/czgfx/work/`), which mounts the build disk `libdisk/CZGFXLIB.DSK` writable as `B:`. From a clean `A0>`, with the sources in CR/LF:

```
A0>R *.*                  (the sources, from software/m80/czgfx/src)
A0>SUBMIT CZBUILD
```

`CZBUILD.SUB`:

- assembles each module with M80;
- combines them with LIB80 into `CZGFX.REL`, in single-pass search order: every module before the modules it calls;
- checks the library with `LIB80 CZGFX/U/E`, which prints nothing when no external is left unsatisfied;
- copies `CZGFX.REL`, `CZGFX.INC` and `CZLINK.SUB` onto `B:`;
- assembles `CZCHK.MAC` against the header as published on `B:` and links it. CZCHK expands every macro and references every entry point, so a broken macro, a missing PUBLIC or a wrong module order fails the library build, not the first program.

The build is good when every assembly prints `No Fatal error(s)`, `LIB80 /U` prints nothing, and L80 prints no `?Undefined` or `%Mult. Def.`.

D.1 Adding to the library

- **A routine:** make it PUBLIC in its module, add a macro if it helps (calling it as `NAME##`, never through an EXTRN in the header), and exercise it in `CZCHK.MAC`.
- **A module:** create `CZxxxx.MAC` (INCLUDE CZGFX.INC, its PUBLICs, an EXTRN for each routine it calls), add its M80 line to `CZBUILD.SUB`, and add it to the LIB80 list after every module that calls it and before every module it calls.
- **A font:** draw its glyphs in `tools/mkfont.py`, which writes the font modules; run it, then CR/LF the output.
- Then release the change (chapter 13).

D.2 M80 and L80 notes

- Global names are 6 characters: `LONGNAME1` and `LONGNAME2` collide.
- Every EXTRN is a link request, used or not.
- Use `LOCAL` for labels in a macro, never `SET` for a label that is referenced forward: its value leaks from the previous expansion.
- `CSEG` with no `ORG`: L80 places the program at 0100H.
- Source must have CR/LF line ends; LF-only source assembles to nothing, silently.

Appendix E. Limits

- No routines for the ACRTC's word-address block commands (CPY, SCPY, DRD, DWT, DMOD), WT and MOD, or the read-backs other than the CP and a pixel. `GCOPY` covers copying on the screen. For the rest, send the command yourself with `CZCMD##` or `CZPUTW##` and its parameters with `CZPUTW##`.

- PAINT's four-seed limit on the real chip (9.1).
- Only the overlay's lower half (`MDOLSL` = 0) is used, since czgfx's screen starts at frame word 0.
- Switching the ACRTC's pixel depth while the display runs (GOVL, GFRAME) is fine in altairsim; CADzilla uses no zoom or cursors, but it has not yet been checked on the real board.